

Smart Clinic Management System

Project Study and Viva Preparation Guide

This PDF is prepared for final project discussion in university. It explains architecture, code organization, database design, testing strategy, and how to implement requirement changes quickly during discussion.

Main stack:

- Frontend: React + Vite + Axios + React Router
- Backend: Node.js + Express + MySQL + JWT
- Security: Role-based access control (admin, doctor, patient)

Team mode during viva: one person drives the code, one person explains architecture, one person explains database and queries, one person explains testing, one person handles change requests.

Chapter 1: System Overview and Business Flow	5 (Owner: Person 1)
Chapter 2: Architecture (High-Level)	7 (Owner: Person 1)
Chapter 3: Frontend Code Structure	9 (Owner: Person 2)
Chapter 4: Backend Code Structure	11 (Owner: Person 2)
Chapter 5: API Endpoint Map	13 (Owner: Person 2)
Chapter 6: Database Design	15 (Owner: Person 3)
Chapter 7: SQL and Transactions	17 (Owner: Person 3)
Chapter 8: Security Model	19 (Owner: Person 4)
Chapter 9: Testing Strategy for Viva	21 (Owner: Person 4)
Chapter 10: Change Requests During Discussion	23 (Owner: Person 5)
Chapter 11: Example Live Change Walkthrough	25 (Owner: Person 5)
Chapter 12: Team Study Plan (5 Members)	27 (Owner: All)
Appendix A: Viva Q&A Bank	29 (Owner: All)
Appendix B: Is This System Enough?	31 (Owner: All)

Focused Revision Index per Team Member

Person 1 -> Pages for Chapters 1-2
Person 2 -> Pages for Chapters 3-5
Person 3 -> Pages for Chapters 6-7
Person 4 -> Pages for Chapters 8-9
Person 5 -> Pages for Chapters 10-11

Table of Contents

Page numbers are listed for fast revision.

Chapter 1: System Overview and Business Flow

What problem this system solves

The system digitizes clinic operations: doctor availability management, patient booking, appointment tracking, and role-specific control for clinic admins.

Core roles and permissions

- Admin: manage doctors, slots, appointments dashboard.
- Doctor: view own schedule, complete/cancel appointments.
- Patient: browse doctors, book slot, view/cancel own appointments.

End-to-end flow

- Admin creates doctor profiles and time slots.
- Patient books available slot for a doctor.
- Slot status changes from available to booked.
- Appointment appears in patient, doctor, and admin views.
- Doctor marks appointment completed or cancelled.

Chapter 2: Architecture (High-Level)

Architecture style

Layered web architecture with clear separation between UI, API, and data. Frontend never talks directly to MySQL. It calls backend endpoints. Backend enforces business rules and security.

Frontend layers

- Pages: role dashboards and screens.
- Components: reusable layout, guards, status components.
- State: AuthContext for current user/session.
- API services: centralized Axios calls.

Backend layers

- Routes: endpoint mapping.
- Controllers: request handling and response shaping.
- Middleware: auth, role checks, error handling.
- Data access: MySQL pool and query helper.

Chapter 3: Frontend Code Structure

Important frontend files

- src/App.jsx: route tree and role-based route groups.
- src/components/ProtectedRoute.jsx: blocks unauthorized access.
- src/state/AuthContext.jsx: login/register/logout and session restore.
- src/api/client.js + services.js: API contracts and token injection.

How token is used

JWT token is stored in localStorage after login. Axios interceptor automatically sends Authorization: Bearer <token> in every request.

How role navigation works

After login, routeForRole(role) sends users to their home path. ProtectedRoute also validates allowed roles and redirects wrong role users.

Chapter 4: Backend Code Structure

Important backend files

- src/app.js: app initialization, middleware, route registration.
- src/server.js: DB readiness check + server start.
- src/config/db.js: mysql2 pool and query helper.
- src/middleware/authMiddleware.js: JWT verification and role restriction.

Controller responsibilities

- authController: login/register/me.
- doctorController: list/create/update/deactivate doctor.
- slotController: list/create/delete slot.
- appointmentController: booking transaction + status update.
- dashboardController: aggregated stats for admin.

Chapter 5: API Endpoint Map

Authentication

- POST /api/auth/register
- POST /api/auth/login
- GET /api/auth/me

Doctors and slots

- GET /api/doctors
- POST /api/doctors (admin)
- PUT /api/doctors/:id (admin)
- DELETE /api/doctors/:id (admin deactivate)
- GET /api/slots
- POST /api/slots (admin)
- DELETE /api/slots/:id (admin)

Appointments and dashboard

- GET /api/appointments
- POST /api/appointments (patient booking)
- PATCH /api/appointments/:id/status
- GET /api/dashboard/stats (admin)

Chapter 6: Database Design

Tables

- users: all accounts with role field.
- slots: doctor availability windows.
- appointments: patient booking record linked to slot.

Key relationships

- slots.doctor_id -> users.id
- appointments.patient_id -> users.id
- appointments.doctor_id -> users.id
- appointments.slot_id -> slots.id (unique)

Important constraints

- Unique email in users.
- Slot has CHECK end_time > start_time.
- One appointment per slot via unique slot_id in appointments.

Chapter 7: SQL and Transactions

Why transaction is required on booking

Two patients may try to book the same slot at the same time. Without transaction, race condition can create duplicate bookings.

Booking transaction logic

- BEGIN TRANSACTION
- SELECT slot FOR UPDATE where status = available
- INSERT appointment
- UPDATE slot status to booked
- COMMIT

Rollback behavior

If any step fails, rollback keeps data consistent.

Chapter 8: Security Model

Security elements implemented

- JWT token signing and verification.
- Route protection middleware.
- Role-based restriction middleware.
- CORS configuration with client URL.
- Helmet for HTTP security headers.

Role rules examples

- Only admin can create doctor and slots.
- Only patient can book appointment.
- Patient can only cancel own appointments.
- Doctor can only update status in own schedule.

Chapter 9: Testing Strategy for Viva

Manual test matrix

- Auth tests: login success/fail, wrong role selection.
- Authorization tests: patient trying admin endpoint should fail 403.
- Booking tests: successful booking changes slot status.
- Concurrency check: same slot cannot be booked twice.
- Status update tests: completed and cancelled flows.

What to say in discussion

Testing was done by role-based scenario validation, negative testing for forbidden actions, and state verification in database-backed screens.

Chapter 10: Change Requests During Discussion

Expected professor style requests

- Add appointment notes mandatory for specific specialization.
- Add search by date range in appointments.
- Add doctor active/inactive toggle in UI.
- Prevent booking within less than 2 hours from current time.

How to implement fast

- Step 1: update backend validation/controller.
- Step 2: update frontend form/service payload.
- Step 3: add DB column/index only if needed.
- Step 4: test role flow impacted by the change.

Chapter 11: Example Live Change Walkthrough

Sample requirement

Requirement: Doctor cannot cancel appointment if it is less than 30 minutes before slot start.

Implementation plan

- Read appointment + slot start_time in update endpoint.
- If status change request is cancelled and requester is doctor, compare current time to start_time.
- Return 400 with clear message if blocked.
- Frontend shows returned message in ErrorMessage component.

Why this reflects software principles

- Encapsulation of business rule in backend domain logic.
- Single source of truth for validation.
- Consistent UX through API error propagation.

Chapter 12: Team Study Plan (5 Members)

Team assignment with chapter ranges

- Person 1 (Presentation lead): Chapters 1-2
- Person 2 (Code structure lead): Chapters 3-5
- Person 3 (Database lead): Chapters 6-7
- Person 4 (Security and testing lead): Chapters 8-9
- Person 5 (Change request lead): Chapters 10-11

Responsibilities in viva

- Person 1: explain problem, value, and architecture diagram verbally.
- Person 2: navigate frontend/backend folders and endpoint flow.
- Person 3: explain schema, keys, constraints, and booking transaction.
- Person 4: explain JWT, middleware, and test cases.
- Person 5: implement requested live change quickly.

Quick revision timing

- Day before: each person reads own chapters twice.
- Final rehearsal: 20-minute demo + 20-minute Q&A simulation.
- Keep one laptop user as driver, one as observer for QA.

Appendix A: Viva Q&A Bank

- Why JWT and not session cookies?
- How do you prevent duplicate booking?
- Why keep users in one table with role column?
- What happens if token is expired?
- How to scale if number of appointments grows x10?
- What tests prove authorization is correct?
- How do you rollback if booking fails halfway?
- How to add notifications feature with minimum changes?

Appendix B: Is This System Enough?

For a university principles-of-software project, this implementation is enough if you demonstrate it cleanly and explain design decisions well. It includes role-based architecture, persistent database, secure auth, business flow coverage, and practical change-readiness.

What makes it acceptable

- Clear separation of concerns.
- Real database design with keys and constraints.
- Role-based authorization rules.
- Reproducible setup and seeded demo data.
- End-to-end tested core use cases.

Optional upgrades if time remains

- Automated backend API tests.
- Pagination and advanced filters.
- Audit log for status changes.

